# Software Design Document

countdown: An X Window Countdown Timer

Countdown Software Project

Prepared by: J. R. Evans

| | |
|---|---|
| Document identifier: | CD-SDD-001 |
| Revision: | 1.0 |
| Date: | June 13, 2026 |
| Status: | Released |
| Governing standard: | IEEE Std 1016-2009 (SDD) |

*This document is prepared in accordance with the principles of rational documentation articulated in D. L. Parnas and P. C. Clements, "A Rational Design Process: How and Why to Fake It," IEEE Transactions on Software Engineering, vol. SE-12, no. 2, pp. 251–257, February 1986.*

# Contents

# 1 Introduction

## 1.1 Purpose

This Software Design Document (SDD) describes how `countdown` is constructed so as to satisfy the requirements of CD-SRS-001. It is the bridge between the SRS and the literate source `countdown.w`: every design decision here is realised by a named CWEB section there.

## 1.2 Scope

The SDD covers the internal structure of the single executable tangled from `countdown.w`: its decomposition into modules, the data it holds, the control flow of its event-and-timer loop, and its use of the Xlib and POSIX interfaces.

## 1.3 Rational-process note

Following Parnas and Clements, the design is presented as a clean decomposition derived from the requirements. The genuine development was iterative; the idealised account is given because it is what a maintainer needs in order to understand and safely change the program.

## 1.4 Design rationale: literate programming

The overriding design constraint (REQ-D-01) is that the program be a literate document. This shapes every other decision: the program is decomposed into many small named sections, each below twenty-four lines (REQ-D-02), so that the woven document reads as a sequence of digestible ideas. The compilable C file is treated strictly as a regenerable artefact (REQ-D-04).

# 2 Design overview

## 2.1 Architectural style

`countdown` is a single-threaded, event-driven program. A central loop multiplexes two stimuli—X events and the passage of time—using one POSIX `select` call, and repaints the window in response to either. All state lives in one record; the routines are pure functions of that record plus the system clock.

## 2.2 Top-level decomposition

The tangled program is assembled in this order (the unnamed root section of `countdown.w`):

1. Header files
2. Constants
3. The application state `App`
4. Parse the target time
5. Compute the remaining seconds (and the run-state toggle)
6. Format a duration

7. Open the display / Create the window / Create the graphics context
8. Render the display
9. Handle one X event
10. The event-and-timer loop
11. The `main` function

### 2.3 Mapping to requirements

| Design element | Requirements satisfied |
|---|---|
| `parse_target` | REQ-F-01, REQ-F-03, REQ-F-04, REQ-T-03 |
| `DEFAULT_TARGET` macro | REQ-F-02 |
| `remaining_secs` | REQ-F-06, REQ-T-02 |
| `fmt_duration` | REQ-F-05 |
| `open_display/create_window/create_gc` | REQ-G-01, REQ-G-06 |
| `render` | REQ-F-05, REQ-F-07, REQ-G-02 |
| `handle_event`, `toggle_run` | REQ-G-03, REQ-G-04, REQ-G-05 |
| `run_loop` | REQ-T-01, REQ-T-04 |
| Many small sections; POSIX section | REQ-D-01–REQ-D-03 |

## 3 Data design

### 3.1 The `App` record

All mutable state is held in one structure, threaded by pointer through every routine. This makes data flow explicit and keeps the routines individually testable.

| Field | Type | Meaning |
|---|---|---|
| `dpy` | `Display *` | Connection to the X server. |
| `win` | `Window` | The top-level window. |
| `gc` | `GC` | Graphics context for text. |
| `font` | `XFontStruct *` | Loaded font, or `NULL`. |
| `screen` | `int` | Default screen number. |
| `target` | `time_t` | The instant counted down to. |
| `running` | `int` | 1 while counting, 0 while paused/armed. |
| `started` | `int` | 1 once the user has begun. |
| `frozen` | `long` | Remaining seconds captured at pause. |

### 3.2 Representation of time

The target is stored as a `time_t` (seconds since the Unix epoch), computed once by `mktime`. The remaining time is recomputed on demand by differencing the target against `time(NULL)`,

so the display can never drift relative to the system clock. Clamping at zero (REQ-F-06) occurs in `remaining_secs`.

# 4 Component design

## 4.1 Target parsing (`parse_target`)

Decomposes a `"YYYY-MM-DD HH:MM:SS"` string with `sscanf`, populates a `struct tm` in local time with `tm_isdst = -1`, and converts it with `mktime`. A field-count mismatch or an unrepresentable date causes a diagnostic and exit 2 (REQ-F-04). Splitting the conversion into a sub-section keeps each below the line limit.

## 4.2 Remaining-time computation (`remaining_secs`)

Returns `max(0, target - now)` as whole seconds, where `now` is `time(NULL)`. The clamp guarantees REQ-F-06.

## 4.3 Duration formatting (`fmt_duration`)

Converts a second count to days plus `HH:MM:SS` into a caller-supplied buffer using the bounds-checked `snprintf`, satisfying REQ-F-05 with no possibility of overflow.

## 4.4 GUI bring-up

`open_display` connects via `XOpenDisplay(NULL)` (failure ⇒ exit 1). `create_window` creates and maps a simple window and selects exposure, key, button, and structure events. `create_gc` builds a graphics context and loads the preferred font, falling back gracefully if it is absent (REQ-G-06).

## 4.5 Rendering (`render`)

Clears the window and draws four lines: the target (via `localtime` and `strftime`), the remaining time, a status line, and a control hint (REQ-G-02). When not running it draws the frozen figure, so paused and armed displays hold still.

## 4.6 Event handling (`handle_event`, `toggle_run`)

Dispatches on event type: expose/configure repaint (REQ-G-05); `q`/Escape request quit; space or mouse click toggle the run state via `toggle_run`, which implements the armed→running, running→paused, and paused→running transitions and captures `frozen` on pause.

## 4.7 The event-and-timer loop (`run_loop`)

The heart of the design. Each iteration first drains all buffered events with `XPending`/`XNextEvent` (necessary because `select` reports only socket-level readability), then blocks on `select` over the X connection descriptor with a quarter-second timeout, then repaints if running. This yields sub-second visual updates (REQ-T-01) with negligible idle CPU (REQ-T-04).

# 5 Control and timing design

## 5.1 Why select, not XNextEvent

A bare `XNextEvent` blocks until input arrives, which would freeze the clock whenever the user sits still. A busy-wait would burn CPU. The chosen design—`select` on the X file descriptor (`ConnectionNumber`) with a timeout—wakes on *either* input or the timer, marrying responsiveness to a steadily ticking display. The `timeval` is rebuilt every iteration because `select` may modify it; an `EINTR` return is benign and simply re-enters the loop.

## 5.2 State machine

| State | Entered by | Display |
|---|---|---|
| Armed | program start | full duration, "ready" |
| Running | space/click from armed or paused | live countdown, "running" |
| Paused | space/click while running | frozen figure, "paused" |
| Arrived | remaining reaches 0 while running | `0 d 00:00:00`, arrival message |

# 6 Interface design

## 6.1 Xlib interface

The program uses only core Xlib (REQ-B-02): `XOpenDisplay`, `XCreateSimpleWindow`, `XStoreName`, `XSelectInput`, `XMapWindow`, `XCreateGC`, `XSetForeground`, `XLoadQueryFont`, `XSetFont`, `XClearWindow`, `XDrawString`, `XFlush`, `XPending`, `XNextEvent`, `XLookupKeysym`, `ConnectionNumber`, and the teardown calls `XFreeFont`, `XFreeGC`, `XDestroyWindow`, `XCloseDisplay`.

## 6.2 POSIX interface

The program rests on a deliberately small set of POSIX services, gathered and documented in the "POSIX system services" section of `countdown.w` and indexed there under *system call* (REQ-D-03): `time` (current time), `mktime` (calendar→instant, with DST), `localtime` (instant→broken-down time for the header), `select` with `FD_ZERO`/`FD_SET` (I/O multiplexing with timeout), and `getenv` (called by Xlib to read `DISPLAY`).

## 6.3 Command-line and environment interface

As specified in CD-SRS-001 §3.6: optional target argument; exit codes 0/1/2; `DISPLAY` and `TZ` environment variables.

## 7 Design constraints and decisions

1. **Module-size limit (REQ-D-02).** Long routines are split into named sub-sections (e.g. "Fill the broken-down time and convert it") so no woven section exceeds twenty-four code lines. The largest section is sixteen lines.
2. **One state record, not globals.** Improves testability and makes the data flow auditable.
3. **Recompute, don't accumulate.** Remaining time is always `target - now`, so the display cannot drift.
4. **Graceful font fallback (REQ-G-06).** A missing preferred font is not fatal.
5. **No toolkit (REQ-B-02).** Keeps the program small, portable, and fully readable.

## 8 Build and packaging design

The `Makefile` drives the CWEB tools through `cweb.sh` (REQ-B-01). `make tangle` runs `ctangle` to produce `countdown.c`; `make compile` builds it with `cc` against Xlib, supplying `-I/-L` for `/opt/X11` on macOS or the default paths on Linux; `make weave` and `make pdf` produce the typeset program. The C, TEX, and PDF outputs are artefacts; the source of record is `countdown.w` (REQ-D-04).